

ML Principles: labo 7

Hassan Haddouchi

Inleiding en doel van de opdracht:

Het doel van deze opdracht het bouwen en trainen van een eenvoudig neurale netwerk om handgeschreven cijfers te classificeren.

Met deze opdracht zal je begrijpen wat de *principles* van een eenvoudig ML model, zonder diep in te gaan op de verschillende componenten van een neurale netwerk.

We zullen de MNIST dataset gebruiken die we al meerdere malen hebben gezien tijdens de theorielessen.

Opdracht

In de laatste 2 labo's heb je voor de eerste keer machinaal geleerd. Dit deden we door middel van lineaire regressie en de mean square error (MSE). Lees zeker de Labo Gids en de toelichting bij de vorige opdracht als je deze 2 termen nog steeds niet begrijpt!

Tijdens de laatste theorieles heb ik toegelicht dat je ook machinaal kan leren door middel van een neurale netwerk. Dat is wat we vandaag zullen doen, namelijk een eenvoudig neurale netwerk ontwikkelen om handgeschreven cijfers uit de MNIST dataset te classificeren. We doen hier dus aan een techniek die we noemen: classificatie.

Wat is classificatie? Classificatie is één van de fundamentele taken binnen ML en AI. Het verwijst naar het proces van het toewijzen van objecten of gegevenspunten aan één of meerdere vooraf gedefinieerde categorieën of klassen op basis van hun kenmerken.

Denk aan het kat-en-hond voorbeeld dat ik vaak heb gebruikt tijdens de theorielessen. Een ML programma dat een onderscheid maakt tussen een foto van een kan en een foto van een hond, gebruikt hiervoor classificatie.

Een classificatieprobleem heeft 2 componenten:

- Kenmerken (features): dit zijn de eigenschappen of variabelen die worden gebruikt om objecten te beschrijven. Bijvoorbeeld, bij het classificeren van handgeschreven cijfers, kunnen de kenmerken de pixelintensiteiten van de afbeeldingen zijn.
- Klassen (classes): dit zijn de vooraf gedefinieerde categorieën waartoe objecten kunnen behoren. Bijvoorbeeld, bij het classificeren van handgeschreven cijfers, zijn de klassen de cijfers 0 tot en met 9.

Het doel van classificatie is om een model te bouwen dat in staat is om nauwkeurig te voorspellen tot welke klasse een nieuw, onbekend gegevenspunt behoort, op basis van de kenmerken ervan. Dit wordt bereikt door het model te trainen op een dataset waarvoor de juiste klassenlabels bekend zijn, zoals de MNIST dataset in deze opdracht. Tijdens de training leert het model patronen in de gegevens te herkennen die het onderscheid maken tussen de verschillende klassen.

Probleemstelling: handgeschreven cijferclassificatie

Stel je voor dat je een dataset hebt met afbeeldingen van handgeschreven cijfers (0 tot en met 9). Je taak is om een machine learning-model te bouwen dat in staat is om handgeschreven cijfers correct te classificeren op basis van de pixelintensiteiten van de afbeeldingen.

Je hebt een dataset met duizenden voorbeelden van handgeschreven cijfers, elk vertegenwoordigd door een afbeelding met bijbehorend label. Het doel is om een classificatiemodel te trainen op deze dataset, zodat het nauwkeurig kan voorspellen welk cijfer wordt weergegeven in een nieuwe, onbekende afbeelding.

Door dit model te bouwen en te trainen, kunnen we automatisch handgeschreven cijfers herkennen, wat tal van toepassingen heeft, zoals optische tekenherkenning (OCR), postcodering, chequeverwerking, enzovoort.

Opdracht

1. Data inladen:

Laad de MNIST dataset in met behulp van TensorFlow.

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten

# Data voorbereiden en inladen
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

2. **Data verkennen** Verken de structuur van de dataset om een beter begrip te krijgen van de kenmerken en labels van de handgeschreven cijfers.

3. **Data voorbereiden** Bereid de dataset voor door de features te normaliseren en de labels te coderen naar one-hot vectors.

```
# Normaliseren van de pixelwaarden van 0-255 naar 0-1
X_train = X_train / 255.0
X_test = X_test / 255.0
```

Het normaliseren van de pixelwaarden, in dit geval van 0-255 naar 0-1 is een veelgebruikte stap bij het voorbereiden van afbeeldingsgegevens voor neurale netwerken. Door de pixelwaarden te normaliseren naar een bereik van 0 tot 1, zorgen we ervoor dat alle inputwaarden van hetzelfde formaat zijn. Dit helpt het model om consistent te leren van de gegevens en voorkomt dat de waarden buiten het verwachte bereik vallen, wat stabiliteit brengt in het trainingsproces.

4. **Bouw een model** Bouw een sequentieel neurale netwerkmodel met behulp van TensorFlow of Keras. Het model moet bestaan uit minstens één inputlaag, één of meerdere verborgen lagen en één uitvoerlaag.

Gebruik de functie `Sequential` om het model te bouwen. Gebruik de functie `Flatten` om de afbeeldingen af te vlakken. Daarna gebruik je de functie `Dense` om een verborgen laag met 128 neuronen en een `ReLU`-

activatie te bouwen. Gebruik dan opnieuw de `Dense` functie om een uitvoerlaag te bouwen met 10 neuronen (één voor elk cijfer) en een `softmax`-activatie.

5. **Model compileren** Gebruik de functie `compile` om uw model te compileren met de `adam` optimizer en een loss functie. Gebruik als `metric` accuracy.
6. **Model trainen** Train het neurale netwerkmodel op de trainingsgegevens. Gebruik enkel de hyperparameters `epochs` en zet deze op 5. Om het model te trainen gebruik je de functie `fit`.
7. **Model evalueren** Evalueer de prestaties van het getrainde model op de testset om de nauwkeurigheid van de classificatie te beoordelen. Gebruik hiervoor de functie `evaluate`.

Hieronder staat de output van mijn classificatie model. Ik heb 5 epochs en mijn model bestaat uit slechts één verborgen laag. Dit verklaart de hoge accuracy, wat mogelijk duidt op overfitting.

Het voorkomen van overfitting is op dit moment buiten de scope van deze opdracht. Later zullen we zien hoe we gelijkaardige problemen rond overfitting kunnen verhelpen.

```
Epoch 1/5
1875/1875 [=====] - 7s 3ms/step - loss: 0.2576 - accuracy: 0.9263
Epoch 2/5
1875/1875 [=====] - 5s 3ms/step - loss: 0.1102 - accuracy: 0.9677
Epoch 3/5
1875/1875 [=====] - 6s 3ms/step - loss: 0.0778 - accuracy: 0.9761
Epoch 4/5
1875/1875 [=====] - 5s 3ms/step - loss: 0.0586 - accuracy: 0.9820
Epoch 5/5
1875/1875 [=====] - 6s 3ms/step - loss: 0.0459 - accuracy: 0.9854
313/313 [=====] - 1s 2ms/step - loss: 0.0699 - accuracy: 0.9766
Test Accuracy: 0.9765999913215637
```